Brief Insights on Performance and Architecture

Apache Spark follows a distributed architecture where the **Driver** coordinates the application, the **Cluster Manager** allocates resources, and the **Executors** perform the actual data processing. This architecture allows Spark to process large datasets efficiently by dividing the work into smaller tasks that run in parallel across multiple executors.

One of Spark's main strengths is **lazy evaluation**. Instead of executing each transformation immediately, Spark records all transformations in a Directed Acyclic Graph (DAG) and optimizes the execution plan before running it. This reduces unnecessary computations and improves overall performance.

Performance also depends on the type of transformation being used. **Narrow transformations**, such as filter() and select(), process data within the same partition and are generally faster. **Wide transformations**, such as groupBy() and join(), require a **shuffle**, where data is exchanged between executors. Since shuffling involves network communication and disk operations, it is more expensive and can increase execution time.

The choice of file format also affects performance. **Parquet** provides better performance than **CSV** because it stores data in a compressed columnar format, preserves schema information, and supports predicate pushdown. This allows Spark to read only the required columns and rows instead of scanning the entire dataset, resulting in faster queries and reduced storage usage.

To achieve better performance, it is recommended to filter data as early as possible, select only the required columns, avoid unnecessary shuffles, and use show() instead of collect() when inspecting large datasets. These practices help reduce memory usage and improve the efficiency of Spark applications.